

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler

from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)
```

```
In [2]: data = pd.read_csv(r"C:\Users\Sanskriti\Downloads\Crop_recommendation.csv")

print(data.shape)
data.head()
```

(2200, 8)

```
Out[2]:
```

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

```
In [3]: print(data.info())
print(data.isnull().sum())
print(data.describe())
```

```

<class 'pandas.DataFrame'>
RangeIndex: 2200 entries, 0 to 2199
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   N                2200 non-null  int64
1   P                2200 non-null  int64
2   K                2200 non-null  int64
3   temperature      2200 non-null  float64
4   humidity          2200 non-null  float64
5   ph                2200 non-null  float64
6   rainfall          2200 non-null  float64
7   label            2200 non-null  str
dtypes: float64(4), int64(3), str(1)
memory usage: 153.0 KB
None
N                0
P                0
K                0
temperature      0
humidity         0
ph               0
rainfall         0
label            0
dtype: int64

```

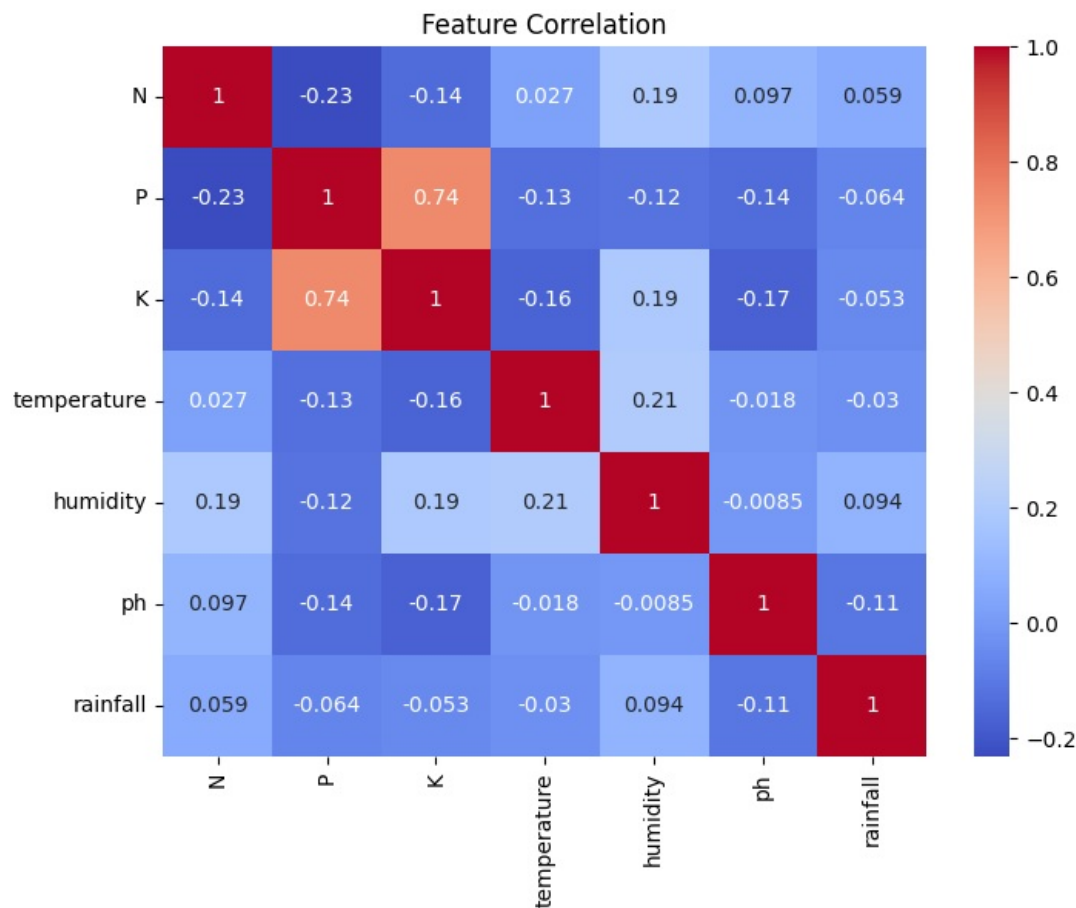
	N	P	K	temperature	humidity	\
count	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	
mean	50.551818	53.362727	48.149091	25.616244	71.481779	
std	36.917334	32.985883	50.647931	5.063749	22.263812	
min	0.000000	5.000000	5.000000	8.825675	14.258040	
25%	21.000000	28.000000	20.000000	22.769375	60.261953	
50%	37.000000	51.000000	32.000000	25.598693	80.473146	
75%	84.250000	68.000000	49.000000	28.561654	89.948771	
max	140.000000	145.000000	205.000000	43.675493	99.981876	

	ph	rainfall
count	2200.000000	2200.000000
mean	6.469480	103.463655
std	0.773938	54.958389
min	3.504752	20.211267
25%	5.971693	64.551686
50%	6.425045	94.867624
75%	6.923643	124.267508
max	9.935091	298.560117

```

In [4]: plt.figure(figsize=(8,6))
sns.heatmap(data.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.title("Feature Correlation")
plt.show()

```



```
In [16]: print(data.describe())
```

	N	P	K	temperature	humidity
count	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000
mean	50.551818	53.362727	48.149091	25.616244	71.481779
std	36.917334	32.985883	50.647931	5.063749	22.263812
min	0.000000	5.000000	5.000000	8.825675	14.258040
25%	21.000000	28.000000	20.000000	22.769375	60.261953
50%	37.000000	51.000000	32.000000	25.598693	80.473146
75%	84.250000	68.000000	49.000000	28.561654	89.948771
max	140.000000	145.000000	205.000000	43.675493	99.981876

	ph	rainfall
count	2200.000000	2200.000000
mean	6.469480	103.463655
std	0.773938	54.958389
min	3.504752	20.211267
25%	5.971693	64.551686
50%	6.425045	94.867624
75%	6.923643	124.267508
max	9.935091	298.560117

```
In [5]: X = data.drop("label", axis=1)
        y = data["label"]
```

```
In [6]: X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )
```

```
In [7]: scaler = StandardScaler()

        X_train_s = scaler.fit_transform(X_train)
        X_test_s = scaler.transform(X_test)
```

```
In [8]: models = {
        "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
        "Decision Tree": DecisionTreeClassifier(random_state=42),
        "Logistic Regression": LogisticRegression(max_iter=1000),
        "KNN": KNeighborsClassifier()
    }

        results = {}
```

```
In [9]: for name in ["Random Forest", "Decision Tree"]:
        model = models[name]
        model.fit(X_train, y_train)
```

```

    pred = model.predict(X_test)

    print("\n", name)
    print("Accuracy:", accuracy_score(y_test, pred))
    print("F1:", f1_score(y_test, pred, average='weighted'))

    results[name] = accuracy_score(y_test, pred)

```

Random Forest
Accuracy: 0.9931818181818182
F1: 0.9931754816901672

Decision Tree
Accuracy: 0.9863636363636363
F1: 0.9863152866630287

```

In [10]: for name in ["Logistic Regression", "KNN"]:
        model = models[name]
        model.fit(X_train_s, y_train)
        pred = model.predict(X_test_s)

        print("\n", name)
        print("Accuracy:", accuracy_score(y_test, pred))
        print("F1:", f1_score(y_test, pred, average='weighted'))

        results[name] = accuracy_score(y_test, pred)

```

Logistic Regression
Accuracy: 0.9636363636363636
F1: 0.9635115059268677

KNN
Accuracy: 0.9568181818181818
F1: 0.9567492608232794

```

In [11]: best_model = max(results, key=results.get)

        print("\nMODEL COMPARISON:")
        for k, v in results.items():
            print(k, ":", round(v, 4))

        print("\n BEST MODEL:", best_model)

```

MODEL COMPARISON:
Random Forest : 0.9932
Decision Tree : 0.9864
Logistic Regression : 0.9636
KNN : 0.9568

BEST MODEL: Random Forest

```

In [12]: rf = RandomForestClassifier(n_estimators=100, random_state=42)
        cv_scores = cross_val_score(rf, X, y, cv=5)

        print("\nCV Scores:", cv_scores)
        print("Mean CV Accuracy:", cv_scores.mean())

```

CV Scores: [0.99772727 0.99318182 0.99772727 0.99545455 0.98863636]
Mean CV Accuracy: 0.9945454545454545

```

In [13]: import joblib

        joblib.dump(rf, "crop_model.pkl")

```

Out[13]: ['crop_model.pkl']

```

In [14]: import os
        print(os.getcwd())

```

C:\Users\Sanskruti

In []: